

ANÁLISIS DE PLATAFORMA · CONFIDENCIAL

Informe de Estandarización de Plataforma

Galaxion (GLX) + SAT · convergencia hacia un estándar común `tocpait-glx`

Proyecto	Zeus · Galaxion
Entorno	GLX SIT / UAT / PROD · SAT dev
Clústeres	<code>tocpait-glx-sit</code> · <code>tocpait-glx-uat</code> · <code>tocpait-glx</code> · <code>tocpait-sat-dev</code>
Recolección	2026-06-23 (GLX) / 2026-06-22 (SAT dev) — solo lectura
Identificación	pa = Panamá
Fecha del informe	2026-06-24

Estado actual → estándar común deseado → acción

Objetivo

Este informe no es solo un inventario del estado actual: para cada dimensión declara **qué está mal, en qué entorno y cuál es la configuración deseada**, de modo que la brecha y la acción queden explícitas.

El propósito es definir un **único estándar de configuración común** hacia el cual converjan Galaxion (SIT/UAT/PROD) y SAT. La orientación es **estado actual** → **estándar común deseado** → **acción**, para armonizar los entornos en lugar de mapearlos por separado.

El objetivo es **un solo estándar** a través de GLX y SAT. El cliente quiere los entornos armonizados, no tratados como mundos separados. SAT ejecuta un conjunto de aplicaciones distinto (`cr-*` / `iuc-*` / `adminnntt-*`), pero le aplican las **mismas reglas de plataforma** (§5).

0. Resumen ejecutivo — qué corregir

Tabla accionable: problema, entorno, acción objetivo y severidad. Los puntos que más importan, ordenados por prioridad.

#	Problema	Entorno	Acción (objetivo)	Severidad
1	Descarga de imágenes fallando — TLS del registry no confiable (x509: certificate signed by unknown authority) → 14 rollouts/migraciones congelados	Galaxion PROD (<code>tocpait-glx</code>)	Restaurar la confianza de la CA de <code>glx-registry.tigo.com.pa</code> en los nodos; alertar sobre <code>ImagePullBackOff</code>	CRÍTICO
2	<code>maildev</code> descargado desde Docker Hub público	UAT + PROD	Replicar al registry interno o eliminar de producción	ALTO
3	Imágenes <code>-SNAPSHOT</code> en producción	PROD	Prohibir tags mutables/SNAPSHOT en UAT/PROD; promover releases inmutables	ALTO
4	Sin autoescalado / sin HA (1 réplica en todos lados)	Los 4	HPA para APIs stateless, KEDA para consumidores de cola, <code>minReplicas ≥ 2</code>	ALTO
5	Istio instalado pero sin uso	GLX SIT	Elegir un único estándar de malla para todos los entornos (quitar de SIT o adoptar en todos)	MED
6	Acoplamiento de registry (UAT descarga del registry de SIT)	SIT + UAT	Registry por entorno o pipeline de promoción claro	MED
7	Observabilidad inconsistente (Prometheus apagado en SIT; logging de PROD desconocido)	SIT + PROD	Prometheus+Grafana común + logging unificado en GLX/SAT	MED
8	Cargas de trabajo en el namespace <code>argocd</code>	SAT	Namespace de aplicación dedicado	MED
9	Errores tipográficos en hosts de ingress (<code>.cam</code> , host de entorno equivocado)	Todos	Corregir los hosts a <code>*.tigo.com.pa</code>	BAJO
10	Sobre-asignación de CPU en SIT (500m vs 150m)	SIT	Ajustar al perfil de UAT/PROD	BAJO
11	Deriva del conjunto de aplicaciones entre entornos	SIT/UAT/PROD	Reconciliar el conjunto de componentes por entorno	BAJO

1. Comparación de plataforma (estado actual)

Dimensión	GLX SIT	GLX UAT	GLX PROD	SAT dev
Clúster	tocpait-glx-sit	tocpait-glx-uat	tocpait-glx	tocpait-sat-dev
Distribución K8s	RKE2 v1.33.0+rke2r1	RKE2 v1.33.0+rke2r1	RKE2 v1.33.5+rke2r1	RKE2 v1.32.8+rke2r1 — ⚠ versiones de nodo dispares (1.28.15 / 1.32.8 / 1.34.1)
Topología	3 mstr + 6 wrk + 2 spark (11)	3 mstr + 10 wrk + 2 spark (15)	3 mstr + 22 wrk (z1/z2) + 2 spark (27)	3 mstr + pools dedicados: 2 NFS, 3 RabbitMQ, 1 SLB, 1 jobs, 4 ws-node (~14); 2 nodos acordonados
Namespace de app	glx (~78)	glx (~77)	glx (~77)	argocd (compartido — anti-patrón), 45
Portafolio de apps	Galaxion	Galaxion	Galaxion	NTT/satélite (cr-* / iuc- * / adminntt-*)
Istio	Instalado 1.26.1, sidecar 2/2, sin CRs de malla	No instalado	No instalado	No instalado
Kiali	Ausente	Ausente	Ausente	Ausente
Autoescalado	solo HPA de istiod	ninguno	ninguno	45 HPAs, todos min=max=1
Réplicas	1	1	1	1 (canaries 0/0)
Prometheus/Grafana	Deshabilitado (0/0)	En ejecución	En ejecución	sin Prometheus; Grafana (Loki)
Logging	no observado	ns logging (fluent-bit)	no observado	Loki + Promtail + Grafana
Registry	glx-registry-sit	glx-registry-sit (compartido)	glx-registry (propio)	registry-local
Ingress	nginx glx-<app>- sit... (10.30.15.x)	nginx glx- <app>-uat... (10.30.19.x)	nginx glx- <app>... (10.30.14.x)	nginx host único k8s-dev- sat...

2. Estado actual vs deseado — el estándar común

Para cada dimensión: **qué está mal** → **dónde** → **la configuración deseada (estándar)**. Los fragmentos de configuración muestran el *objetivo*, no los manifiestos completos.

2.1 Autoescalado y resiliencia — el punto esencial

Esto se trata de **autoescalado**, que es distinto de la configuración de recursos del pod en §2.2. Hoy nada escala y nada tiene alta disponibilidad.

Qué está mal	Dónde	Estándar deseado
Sin HPA en absoluto → no puede escalar	GLX SIT/UAT/PROD	HPA real en servicios stateless
45 HPAs pero <code>min=max=1</code> (decorativos)	SAT dev	Reemplazar con min/max reales o KEDA
Réplica única → punto único de fallo	Los 4	<code>minReplicas ≥ 2</code> (línea base de HA)
Consumidores de cola/ETL nunca escalan según la carga	conectores GLX, consumidores SAT	KEDA sobre la profundidad de la cola

Deseado — API/servicio stateless (HPA):

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 2          # línea base de HA – hoy: 1
  maxReplicas: 6
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 65 } }
```

Deseado — consumidor orientado a eventos (KEDA sobre RabbitMQ): para los consumidores `adminntt` de SAT (email/sms/notification/ondemand) y los conectores GLX de payment/mediation/workflow.

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
spec:
  minReplicaCount: 1
  maxReplicaCount: 10
  triggers:
    - type: rabbitmq
      metadata: { queueName: mail_queue, mode: QueueLength, value: "20" }
```

2.2 Recursos del pod (right-sizing)

Esto es la **configuración del pod** (requests/limits). En la revisión se notó que se estaba mostrando *en lugar* de el autoescalado. Ambos importan; manténgalos separados.

Qué está mal	Dónde	Estándar deseado
CPU sobre-asignada (core/backend 500m, ~98% ocioso)	GLX SIT	Alinear al perfil de UAT/PROD (≈150m)
CPU sobre-asignada (100m × 45 = 4.5 cores, ~76m usados)	SAT	Solicitar 25–50m donde esté ocioso
Memoria infradimensionada: req 256Mi / limit 512Mi, JVM <code>-Xmx512m</code> == limit → riesgo de OOM en 17 svcs	SAT	req 384Mi, limit 768Mi, <code>-Xmx</code> ≈ 70% del limit
<code>maildev</code> sin requests/limits (BestEffort)	UAT/PROD	Establecer requests/limits mínimos

Deseado — servicio Java:

```
resources:
  requests: { cpu: 50m, memory: 384Mi } # SAT hoy: 100m / 256Mi
  limits:   { cpu: 500m, memory: 768Mi } # SAT hoy: ... / 512Mi
env:
  - name: JAVA_TOOL_OPTIONS
    value: "-Xmx512m" # ≈70% del limit de 768Mi (hoy -Xmx512m == limit → sin holgura)
```

2.3 Malla de servicios (Istio)

Qué está mal	Dónde	Estándar deseado
Istio instalado, sidecar en cada pod, pero cero CRs de malla, mTLS permissive, sin Kiali → costo, sin beneficio	solo GLX SIT	Un único estándar para todos los entornos

Elija una y aplíquela a los cuatro entornos:

- **Opción A (recomendada, bajo esfuerzo):** quitar la inyección no utilizada de SIT → paridad con UAT/PROD/SAT (sin malla).
- **Opción B (mayor valor):** adoptar Istio en todos lados con **mTLS STRICT + Kiali + políticas base**.

El medio-estado actual (malla en un entorno, sin hacer nada) es el único resultado a evitar.

2.4 Registry e imágenes

Qué está mal	Dónde	Estándar deseado
Las nuevas descargas de imágenes fallan – certificado TLS del registry no confiable (x509), ver §3	PROD	Restaurar la confianza de la CA del registry en los nodos; alertar sobre <code>ImagePullBackOff</code>
<code>maildev</code> desde Docker Hub público	UAT + PROD	Replicar internamente o eliminar de producción
Tags <code>-SNAPSHOT</code> (mutables) en producción	PROD (<code>customer-pa-management</code> , <code>tigo-pa-adjustment-custom-service</code>)	Solo tags de release inmutables; prohibir <code>*-SNAPSHOT</code> en UAT/PROD
UAT descarga desde el registry de SIT	SIT + UAT	Registry por entorno o pipeline de promoción explícito
PROD adelantado respecto a entornos inferiores (<code>geographic-management</code> , <code>tigo-ar-workflow-connector</code>)	mixto	Forzar la dirección de promoción SIT → UAT → PROD

2.5 Observabilidad

Qué está mal	Dónde	Estándar deseado
Prometheus/Grafana escalados a 0/0	GLX SIT	Prometheus + Grafana activos en cada entorno
No se observa el namespace <code>logging</code>	GLX PROD	Confirmar + estandarizar el envío de logs
Tres enfoques de logging distintos (ninguno / fluent-bit→ES / Loki)	GLX vs SAT	Un único stack de logging unificado en GLX + SAT

2.6 Aislamiento de namespaces e higiene de ingress

Qué está mal	Dónde	Estándar deseado
Las cargas de trabajo comparten el namespace <code>argocd</code> (sin aislamiento de radio de impacto)	SAT	Namespace de aplicación dedicado
Error tipográfico de TLD <code>.cam</code>	todos (<code>...tasks-pa-stg.tigo.cam</code> , <code>glx-users-management.tigo.cam</code>)	<code>*.tigo.com.pa</code>
El ingress de UAT apunta a un host <code>sit</code>	UAT (<code>glx-users-management-sit.tigo.cam</code>)	Corregir al host de UAT
Error tipográfico en nombre de recurso <code>equipments-seravice-ingress</code>	todos	Renombrar

3. Incidente — Galaxion PROD (`tocpait-glx`): descarga de imágenes fallando por confianza TLS del registry

CRÍTICO Incidente en PRODUCCIÓN — causa raíz: confianza de CA / certificado TLS (no es imagen ausente)

Entorno: clúster de Galaxion **PRODUCCIÓN** `tocpait-glx`, namespace `glx`, registry `glx-registry.tigo.com.pa`. Inicio ~2026-06-18 (errores con ~5 días de antigüedad al 2026-06-23). **14 pods** afectados.

Causa raíz (confirmada vía `kubectl describe`): los nodos no pueden verificar el certificado TLS del registry de producción —

`tls: failed to verify certificate: x509: certificate signed by unknown authority`.

Por lo tanto, **toda nueva descarga de imagen desde `glx-registry.tigo.com.pa` falla**. Esto es un **problema de confianza de CA / certificado del registry — NO son imágenes ausentes**. Las imágenes existen; los nodos simplemente ya no confían en el certificado del registry (lo más probable: el certificado fue renovado/rotado sin distribuir la CA emisora a los nodos / a la configuración de registry de containerd).

Evidencia (verbatim, desde el clúster):

```
Failed to pull image "glx-registry.tigo.com.pa/glxf-team/itsf-dbmdl:1.1.0-SNAPSHOT":  
Head "https://glx-registry.tigo.com.pa/v2/glxf-team/itsf-dbmdl/manifests/1.1.0-SNAPSHOT":  
tls: failed to verify certificate: x509: certificate signed by unknown authority
```

```
Failed to pull image "glx-registry.tigo.com.pa/galaxion-docker/core/backend/collections-service:8.1.2":  
... tls: failed to verify certificate: x509: certificate signed by unknown authority
```

Mecanismo: los pods ya en ejecución siguen sirviendo porque su imagen está **en caché en el nodo**; solo los **nuevos** pods necesitan descargar. Así, la ola golpea a (a) los helpers de migración de BD de un solo uso `*-dbmdl` y (b) los nuevos ReplicaSets — mientras el Deployment todavía muestra el RS **antiguo** en `1/1`, enmascarando un rollout que silenciosamente nunca se completa. Nótese que `account-receivable-facade` (6.4.0) y `payments-ui` (1.5.3) fallan con el **mismo tag** que ya está corriendo — confirmando que es la ruta de descarga/TLS, no la versión de la imagen.

Afectados (14):

Pod	Imagen	Estado / error
itsf-dbmdl-*	.../glx/itsf-team/itsf-dbmdl:1.1.0-SNAPSHOT	ImagePullBackOff — x509 unknown authority
collections-service-dbmdl-8.1.2-*	.../core/backend/collections-service:8.1.2	ImagePullBackOff — x509 unknown authority
account-receivable-facade-* (RS nuevo)	.../core/backend/account-receivable-facade:6.4.0	ImagePullBackOff (RS antiguo 6.4.0 sigue 1/1)
payments-ui-* (RS nuevo)	.../core/frontend/payments-ui:1.5.3	ErrImagePull (RS antiguo sigue 1/1)
cdr-usage-consumption-service-*	.../eirie/backend/cdr-usage-consumption-service:2.0.4	ImagePullBackOff, 138 reinicios
sms-sender-service-* (RS nuevo)	.../core/backend/sms-sender-service	ImagePullBackOff (RS antiguo sigue 1/1)
+ 8 helpers *-dbmdl (add-ons, addresses, appointments, customer-history, discounts, order-status, otp-verification, security-questions)	varios ...:<versión>	ImagePullBackOff — x509

Todos los fallos son contra el **mismo host de registry** `glx-registry.tigo.com.pa` con el **mismo error x509**.

Acción: (1) **restaurar la confianza de la CA de** `glx-registry.tigo.com.pa` **en los nodos** — agregar la CA del registry a RKE2/containerd (`/etc/rancher/rke2/registries.yaml`) o al almacén de confianza del SO, o reemitir el certificado del registry con una cadena en la que los nodos ya confíen; (2) verificar que las descargas se recuperen (los nuevos pods se programan); (3) **alertar sobre** `ImagePullBackOff` para que un cambio de certificado/registry no pueda volver a congelar silenciosamente los rollouts de producción durante días.

4. Mapeo por aplicación — GLX (apps bajo nuestra responsabilidad)

Constante hoy (todos los entornos GLX): réplica = 1, sin HPA (estado objetivo en §2.1). SIT añade un sidecar istio-proxy (2/2).

4.1 Presencia y tags de imagen (SIT x UAT x PROD)

Registry: SIT/UAT = `glx-registry-sit.tigo.com.pa`, PROD = `glx-registry.tigo.com.pa`.

App	SIT	UAT	PROD	Nota
acquisition-prospects-service	5.1.1	5.1.1	4.5.4	
adjustments-service	8.1.0	8.1.0	7.3.0	
barrings-service	4.3.2	4.3.2	4.3.2	alineado
billing-service	2.4.0	2.4.0	2.2.0	
case-management-pa-audit-worker	release-4.3.0	release-4.3.0	release-4.3.0	⚠ worker expuesto vía ingress
case-management-pa-users-worker	release-4.0.1	release-4.0.1	release-4.0.1	⚠ worker expuesto vía ingress
collections-service	8.1.8	8.1.2	7.3.3	promoción atascada en PROD (§3)
customer-pa-management	release-1.0.0	release-1.0.0	1.0.0-SNAPSHOT	⚠ SNAPSHOT en producción
email-sender-service	3.4.0	3.4.0	3.2.0	
geographic-management	1.0.1-SNAPSHOT	1.0.1-SNAPSHOT	1.0.3	⚠ producción adelantada + SNAPSHOT abajo
maildev	ausente	maildev/maildev	maildev/maildev	⚠ Docker Hub público
otp-verification-service	6.1.0	6.1.0	5.3.0	
prospect-lead	2.0.0	release-1.1.0	1.1.0	⚠ el esquema de tags difiere
sms-sender-service	3.3.0	3.3.0	3.3.0	alineado; RS nuevo atascado (§3)
tigo-ar-workflow-connector	1.0.0	1.0.0	1.0.1	⚠ producción más nueva
tigo-collection-management-workflow-connector	ausente	1.0.0-SNAPSHOT	ausente	solo en UAT
tigo-equipments-workflow-connector	1.12.0	1.10.1	1.8.1	
tigo-pa-adjustment-custom-service	1.0.0-SNAPSHOT	release-3.1.24	1.0.0-SNAPSHOT	⚠ inconsistente en 3 vías
tigo-pa-order-processor	1.2.0	1.2.0	1.1.0	
user-management-workflow-connector	ausente	2.0.1	2.0.1	falta en SIT
workflow-engine-facade	3.1.0	2.3.0	2.1.0	

4.2 Recursos (SIT vs UAT=PROD)

UAT y PROD comparten dimensionamiento idéntico; SIT es el caso atípico (sobre-asigna CPU en core/backend). Se conservan las cifras completas por app de la recolección; el objetivo de estandarización está en §2.2.

Perfil	SIT	UAT = PROD
core/backend (ej. acquisition, email-sender, sms-sender)	500m/1 · 1Gi/2Gi	150m/1 · 768Mi/2Gi
PA/custom (ej. otp, geographic, ar-connector)	50m/500m · 256Mi/1Gi	150m/500m · 768Mi/1Gi
guaranteed (billing-service, workflow-engine-facade)	req == limit	req == limit

5. SAT bajo el estándar común

SAT dev es un portafolio de aplicaciones diferente (POS/caja `cr-*`, IUC/SAP `iuc-*`, admin/ETL NTT `adminntt-*`, `digital-sale-closure`), **pero le aplica el mismo estándar**. SAT no es un stack separado: forma parte de la misma convergencia GLX + SAT. Hoy diverge en cada eje:

Estándar (§2)	SAT actual	Objetivo
Autoescalado	45 HPAs decorativos (<code>min=max=1</code>)	HPA (stateless) + KEDA (los consumidores <code>adminntt/payment</code>); <code>minReplicas ≥ 2</code>
Recursos	CPU 100m (~98% ocioso), mem 256/512 + <code>-Xmx512m ==limit</code> (OOM en 17 svcs)	right-size según §2.2
Namespace	cargas de trabajo en el ns <code>argocd</code>	namespace dedicado
Registry/imágenes	imágenes base desde <code>registry-local</code>	mismas reglas de promoción/inmutabilidad
Observabilidad	solo Loki	stack unificado con GLX

Elementos específicos de SAT a converger:

- **Versiones de nodo dispares** — el clúster mezcla versiones de RKE2/K8s entre nodos (`v1.28.15` , `v1.32.8` , `v1.34.1`) con el plano de control en `v1.32.8` , y **2 nodos acordonados** (`SchedulingDisabled : nfs-01` en el viejo 1.28.15, `wsnode-04` en 1.34.1). Los clústeres GLX son uniformes por entorno (1.33.0 / 1.33.5). → alinear versiones de nodo; drenar/reemplazar los rezagados.
- **PVCs NFS vacíos** — los 45 servicios montan un PVC NFS RWX de 10Gi que está **sin usar** (~450 GiB asignados para nada) y una dependencia dura de NFS sin beneficio → quitar del overlay base.
- **Mala configuración de split-vhost de RabbitMQ** (`/` vs `/sat`) deja mensajes NTT de create/close atascados sin consumidor; **DLQ/backlog sin drenador** (3 SMS muertos, 2 emails atascados).
- **Logging no apto para soporte** — DEBUG efectivo a pesar de `LOG_LEVEL=INFO` , ~100% de ruido de sondas `/health` , sin correlation IDs, sin JSON.
- Autenticación duplicada (un servicio de auth por dominio).

Detalle: `sat-dev-analysis.md` , `sat-observability-logging-report.md` , `sat-dev-observability-plan.md` .

6. Plan de acción (priorizado)

1. **CRÍTICO** **TLS del registry de PROD (§3):** restaurar la CA de `glx-registry.tigo.com.pa` en los nodos (RKE2/containerd o almacén de confianza del SO) para que los 14 rollouts/migraciones congelados se recuperen; añadir una alerta de `ImagePullBackOff` .

2. **ALTO** **Gobierno de imágenes:** eliminar/replicar `maildev`; prohibir `-SNAPSHOT` y referencias públicas en UAT/PROD; forzar la promoción SIT→UAT→PROD.
3. **ALTO** **Autoescalado y HA (§2.1):** introducir HPA (stateless) + KEDA (consumidores), `minReplicas ≥ 2`; reemplazar los HPAs decorativos de SAT.
4. **MED** **Decisión de malla (§2.3):** elegir un único estándar de Istio para todos los entornos.
5. **MED** **Desacoplamiento de registry (§2.4) y paridad de observabilidad (§2.5)** (Prometheus activo en SIT; confirmar logging de PROD; unificar GLX+SAT).
6. **MED** **Namespace de SAT** fuera de `argocd`; quitar los PVCs NFS vacíos; corregir el split-vhost de RabbitMQ + drenador de DLQ.
7. **BAJO** **Right-size de CPU en SIT (§2.2),** corregir errores tipográficos de ingress (§2.6), reconciliar la deriva del conjunto de aplicaciones.